

Decision by Committee: Ensemble Learning

The old saying has it that two heads are better than one. Which naturally leads to the idea that even more heads are better than that, and ends up with decision by committee, which is famously useless for human activities (as in the old joke that a camel is a horse designed by a committee). For machine learning methods the results are rather more impressive, as we'll see in this chapter.

The basic idea is that by having lots of learners that each get slightly different results on a dataset—some learning certain things well and some learning others—and putting them together, the results that are generated will be significantly better than any one of them on its own (provided that you put them together well... otherwise the results could be significantly worse). One analogy that might prove useful is to think about how your doctor goes about performing a diagnosis of some complaint that you visit her with. If she cannot find the problem directly, then she will ask for a variety of tests to be performed, e.g., scans, blood tests, consultations with experts. She will then aggregate all of these opinions in order to perform a diagnosis. Each of the individual tests will suggest a diagnosis, but only by putting them together can an informed decision be reached.

Figure 13.1 shows the basic idea of **ensemble learning**, as these methods are collectively called. Given a relatively simple binary classification problem and some learner that puts an ellipse around a subset of the data, combining the ellipses can provide a considerably more complex decision boundary.

There are then only a couple of questions to ask: which learners should we use, how should we ensure that they learn different things, and how should we combine their results? The methods that we are investigating in this chapter can use any classifier at all. Although in general they only use one type of classifier at a time, they do not have to. A common choice of classifier is the decision tree (see Chapter 12).

Ensuring that the learners see different things can be performed in different ways, and it is the primary difference between the algorithms that we shall see. However, it can also come about naturally depending upon the application area. Suppose that you have lots and lots of data. In that case you could simply randomly partition the data and give different sets of data to different classifiers. Even here there are choices: do you make the partitions separate, or include overlaps? If there is no overlap, then it could be difficult to work out how to combine the classifiers, or it might be very simple: if your doctor always asks for opinions from two colleagues, one specialising in heart problems and one in sports injuries,

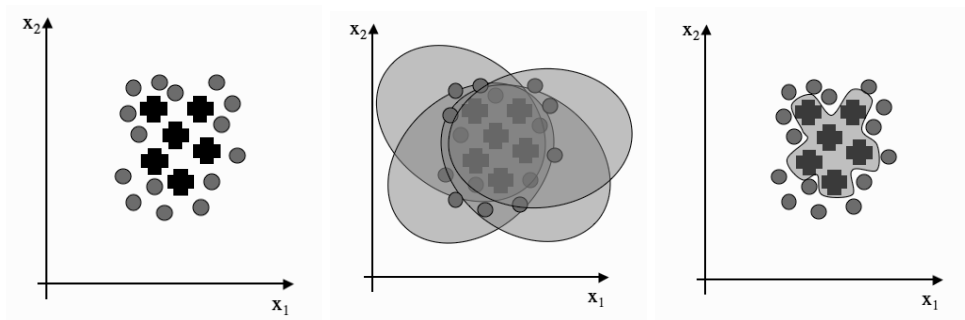


FIGURE 13.1 By combining lots of simple classifiers (here that simply put an elliptical decision boundary onto the data), the decision boundary can be made much more complicated, enabling the difficult separation of the pluses from the circles.

then upon discovering that your leg started hurting after you went for a run she would likely accord more weight to the diagnosis of the sports injury expert.

Interestingly, ensemble methods do very well when there is very little data as well as when there is too much. To see why, think cross-validation (Section 2.2.2). We used cross-validation when there was not enough data to go around, and trained lots of neural networks on different subsets of the data. Then we threw away most of them. With an ensemble method we keep them all, and combine their results in some way. One very simple way to combine the results is to use majority voting — if it's good enough for electing governments in elections, it's good enough for machine learning. Majority voting has the interesting property that for binary classification, the combined classifier will only get the answer wrong if more than half of the classifiers were wrong. Hopefully, this isn't going to happen too often (although you might be able to think of government elections where this has been the case in your view). There are alternative ways to combine the results, as we'll discuss. These things will become clearer as we look at the algorithms, so let's get started.

13.1 BOOSTING

At first sight the claim of the most popular ensemble method, **boosting**, seems amazing. If we take a collection of very poor (**weak** in the jargon) learners, each performing only just better than chance, then by putting them together it is possible to make an **ensemble** learner that can perform arbitrarily well. So we just need lots of low-quality learners, and a way to put them together usefully, and we can make a learner that will do very well.

The principal algorithm of boosting is named AdaBoost, and is described in Section 13.1.1. The algorithm was first described in the mid-1990s by Freund and Shapiro, and while it has had many variations derived from it, the principal algorithm is still one of the most widely used. The algorithm was proposed as an improvement on the original 1990 boosting algorithm, which was rather data hungry. In that algorithm, the training set was split into three. A classifier was trained on the first third, and then tested on the second third. All of the data that was misclassified during that testing was used to form a new dataset, along with an equally sized random selection of the data that was correctly classified. A second classifier was trained on this new dataset, and then both of the classifiers were tested on the final third of the dataset. If they both produced the same output, then that datapoint was ignored, otherwise the datapoint was added to yet another new

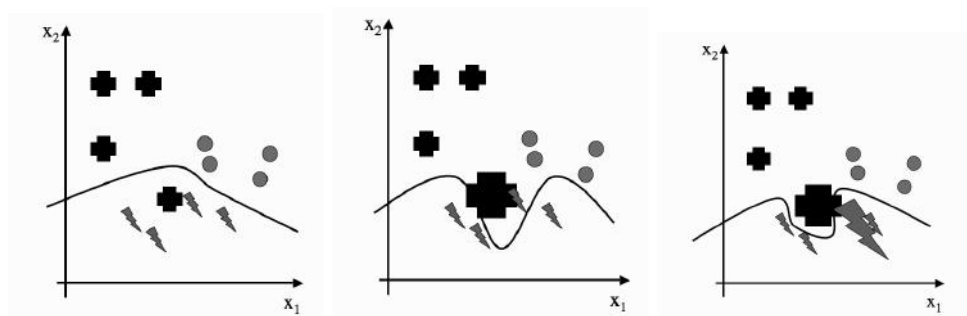


FIGURE 13.2 As points are misclassified, so their weights increase in boosting (shown by the datapoint getting larger), which makes the importance of those datapoints increase, making the classifiers pay more attention to them.

dataset, which formed the training set for a third classifier. Rather than looking further at this version, we will look at the more common algorithm.

13.1.1 AdaBoost

The innovation that AdaBoost (which stands for **adaptive boosting**) uses is to give weights to each datapoint according to how difficult previous classifiers have found to get it correct. These weights are given to the classifier as part of the input when it is trained.

The AdaBoost algorithm is conceptually very simple. At each iteration a new classifier is trained on the training set, with the weights that are applied to the training set for each datapoint being modified at each iteration according to how successfully that datapoint has been classified in the past. The weights are initially all set to the same value, $1/N$, where N is the number of datapoints in the training set. Then, at each iteration, the error (ϵ) is computed as the sum of the weights of the misclassified points, and the weights for incorrect examples are updated by being multiplied by $\alpha = (1 - \epsilon)/\epsilon$. Weights for correct examples are left alone, and then the whole set is normalised so that it sums to 1 (which is effectively a reduction in the importance of the correctly classified datapoints). Training terminates after a set number of iterations, or when either all of the datapoints are classified correctly, or one point contains more than half of the available weight.

Figure 13.2 shows the effect of weighting incorrectly classified examples as training proceeds, with the size of each datapoint being a measure of its importance. As an algorithm this looks like (where $I(y_n \neq h_t(x_n))$ is an indicator function that returns 1 if the target and output are not equal, and 0 if they are):

AdaBoost Algorithm

- Initialise all weights to $1/N$, where N is the number of datapoints
- While $0 < \epsilon_t < \frac{1}{2}$ (and $t < T$, some maximum number of iterations):
 - train classifier on $\{S, w^{(t)}\}$, getting hypotheses $h_t(x_n)$ for datapoints x_n
 - compute training error $\epsilon_t = \sum_{n=1}^N w_n^{(t)} I(y_n \neq h_t(x_n))$
 - set $\alpha_t = \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
 - update weights using:

$$w_n^{(t+1)} = w_n^{(t)} \exp(\alpha_t I(y_n \neq h_t(x_n))) / Z_t, \quad (13.1)$$

where Z_t is a normalisation constant

- Output $f(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$
-

There is nothing too difficult to the implementation, either, as can be seen from the main loop here:

```
for t in range(T):
    classifiers[:,t] = train(data,classes,w[:,t])
    outputs,errors = classify(data,classifiers[0,t],classifiers[1,t])

    index[:,t] = errors
    print "index: ", index[:,t]
    e[t] = np.sum(w[:,t]*index[:,t])/np.sum(w[:,t])

    if t>0 and (e[t]==0 or e[t]>=0.5):
        T=t
        alpha = alpha[:,t]
        index = index[:,t]
        w = w[:,t]
        break

    alpha[t] = np.log((1-e[t])/e[t])
    w[:,t+1] = w[:,t]* np.exp(alpha[t]*index[:,t])
    w[:,t+1] = w[:,t+1]/np.sum(w[:,t+1])
```

Most of the work of the algorithm is done by the classification algorithm, which is given new weights at each iteration. In this respect, boosting is not quite a stand-alone algorithm: the classifiers need to consider the weights when they perform their classifications. It is not always obvious how to do this for a particular classifier, but we have seen methods of doing it for a few classifiers. For the decision tree we saw a method in Section 12.3.1,

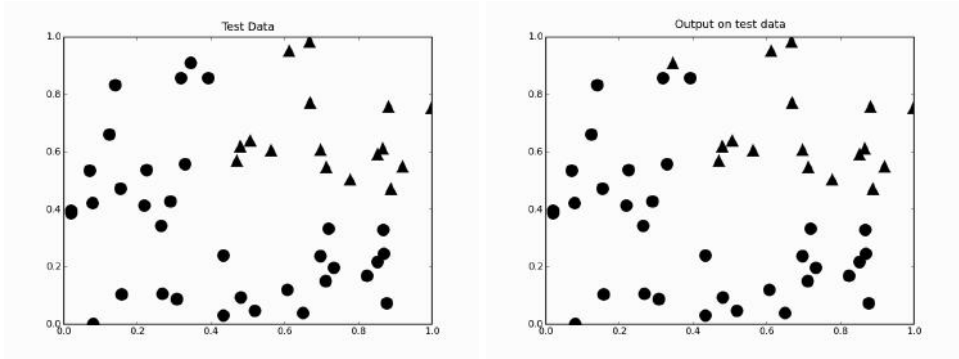


FIGURE 13.3 Boosting learns this simple dataset very successfully, producing an ensemble classifier that is rather more complicated than the simple horizontal or vertical line classifier that the algorithm boosts. On the independent test set shown here, the algorithm gets only 1 datapoint wrong, and that is one that is coincidentally close to one that was misclassified to simulate noise in the training data.

when we looked at the Gini impurity. There, we allowed for a λ matrix that encoded the risks associated with misclassification, and these are a perfect place in which to introduce weights. Modification of the decision tree algorithm to deal with these weights is suggested as an exercise for this chapter. A similar argument can be used for the Bayes' classifier; this was discussed in Section 2.3.1.

As a very simple example showing how boosting works, a very simple classifier was created that can only separate data by fitting one either horizontal or vertical line, with it choosing which to fit at the current iteration at random. A two-dimensional dataset was created with data in the top right-hand corner being in one class, and the rest in another, plus a couple of the datapoints were randomly mislabelled to simulate noise. Clearly, this dataset cannot be separated by a single horizontal or vertical decision boundary. However, Figure 13.3 shows the output of the classifier on an independent test set, where the algorithm gets only one datapoint wrong, and that is one that is coincidentally close to one of the 'noisy' datapoints in the training data. Figure 13.4 shows the training data, the error curve on both the training and testing sets, and the first few iterations of the classifier, which can only put in one horizontal or linear classification line.

Clearly, such impressive results require some explanation and understanding. The key to this understanding is to compute the **loss function**, which is simply the measure of the error that is applied (we have been using a sum-of-squares loss function for many algorithms in the book). The loss function for AdaBoost has the form

$$G_t(\alpha) = \sum_{n=1}^N \exp(-y_n(\alpha h_t(x_n) + f_{t-1}(x_n))), \quad (13.2)$$

where $f_{t-1}(x_n)$ is the sum of the hypotheses of that datapoint from the previous iterations:

$$f_{t-1}(x_n) = \sum_{\tau=0}^{t-1} \alpha_\tau h_\tau(x_n). \quad (13.3)$$

Exponential loss functions are well behaved and robust to outliers. The weights $w^{(t)}$

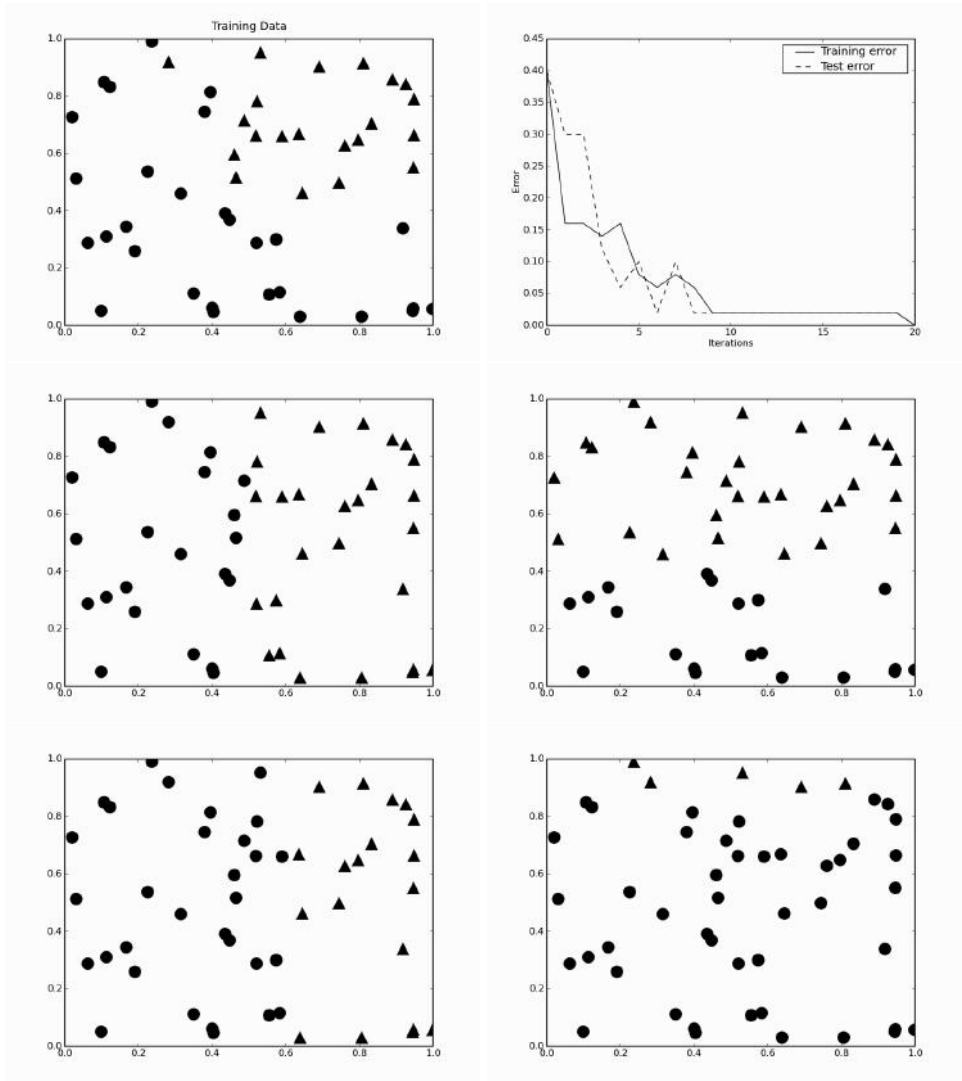


FIGURE 13.4 *Top:* the training data and the error curve. *Middle and bottom:* The first few iterations of the classifier; each plot shows the output of one of the weak classifiers that are boosted by the algorithm.

in the algorithm are nothing more than the second term in Equation (13.2), which can therefore be rewritten as:

$$G_t(\alpha) = \sum_{n=1}^N w^{(t)} \exp(-y_n \alpha h_t(x_n)). \quad (13.4)$$

Deriving the rest of the algorithm from here requires substituting in for the hypotheses h and then solving for α , which produces the full algorithm. Interestingly, this is not the way that AdaBoost was created; this understanding of why it works so well came later. It is possible to choose other loss functions, and providing that they are differentiable they will provide useful boosting-like algorithms, which are collectively known as **arcing** algorithms (for **adaptive reweighting and combining**).

AdaBoost can be modified to perform regression rather than classification (known as **real adaboost**, or sometimes **adaboost.R**). There is another variant on boosting (also called **AdaBoost**, confusingly) that uses the weights to sample from the full dataset, training on a sample of the data rather than the full weighted set, with more difficult examples more likely to be in the training sample. This is more in line with the original boosting algorithm, and is obviously faster, since each training run has fewer data to learn about.

13.1.2 Stumping

There is a very extreme form of boosting that is applied to trees. It goes by the descriptive name of **stumping**. The stump of a tree is the tiny piece that is left over when you chop off the rest, and the same is true here: stumping consists of simply taking the root of the tree and using that as the decision maker. So for each classifier you use the very first question that makes up the root of the tree, and that is it. Often, this is worse than chance on the whole dataset, but by using the weights to sort out when that classifier should be used, and to what extent, as opposed to the other ones, the overall output of stumping can be very successful. In fact, it is pretty much exactly what the simple example that we saw consisted of.

13.2 BAGGING

The simplest method of combining classifiers is known as **bagging**, which stands for **bootstrap aggregating**, the statistical description of the method. This is fine if you know what a **bootstrap** is, but fairly useless if you don't. A bootstrap sample is a sample taken from the original dataset **with replacement**, so that we may get some data several times and others not at all. The bootstrap sample is the same size as the original, and lots and lots of these samples are taken: B of them, where B is at least 50, and could even be in the thousands. The name bootstrap is more popular in computer science than anywhere else, since there is also a bootstrap loader, which is the first program to run when a computer is turned on. It comes from the nonsensical idea of 'picking yourself up by your bootstraps,' which means lifting yourself up by your shoelaces, and is meant to imply starting from nothing.

Bootstrap sampling seems like a very strange thing to do. We've taken a perfectly good dataset, mucked it up by sampling from it, which might be good if we had made a smaller dataset (since it would be faster), but we still ended up with a dataset the same size. Worse, we've done it lots of times. Surely this is just a way to burn up computer time without gaining anything. The benefit of it is that we will get lots of learners that perform slightly differently, which is exactly what we want for an ensemble method. Another benefit is that estimates of the accuracy of the classification function can be made without complicated

analytic work, by throwing computer resources at the problem (technically, bagging is a **variance reducing algorithm**; the meaning of this will become clearer when we talk about bias and variance in Section 2.5). This is a standard technique in modern statistics; we'll see another example in Chapter 15 when we look at Markov Chain Monte Carlo methods. It is sufficiently common to have inspired the comment that “statistics is defined as the discipline where those that think don't count and those that count don't think.”

Having taken a set of bootstrap samples, the bagging method simply requires that we fit a model to each dataset, and then combine them by taking the output to be the majority vote of all the classifiers. A NumPy implementation is shown next, and then we will look at a simple example.

```
# Compute bootstrap samples
samplePoints = np.random.randint(0,nPoints,(nPoints,nSamples))
classifiers = []

for i in range(nSamples):
    sample = []
    sampleTarget = []
    for j in range(nPoints):
        sample.append(data[samplePoints[j,i]])
        sampleTarget.append(targets[samplePoints[j,i]])
    # Train classifiers
    classifiers.append(self.tree.make_tree(sample,sampleTarget,features))
```

The example consists of taking the party data that was used in Section 12.4 to demonstrate the decision tree, and restricting the trees to stumps, so that they can make a classification based on just one variable. The output of a decision tree that uses the whole dataset for this is not surprising: it takes the two largest classes, and separates them. However, using just stumps of trees and 20 samples, bagging can separate the data perfectly, as this output shows:

```
Tree Stump Prediction
['Party', 'Party', 'Party', 'Party', 'Pub', 'Party', 'Study', 'Study', 2
'Party', 'Study']
Correct Classes
['Party', 'Study', 'Party', 'Party', 'Pub', 'Party', 'Study', 'TV', 'Party', 2
'Study']
Bagged Results
['Party', 'Study', 'Party', 'Party', 'Pub', 'Party', 'Study', 'TV', 'Party', 2
'Study']
```

13.2.1 Subbagging

For some reason, ensemble methods often have good names, such as boosting and bagging (and we will see my choice for best-named, **bragging**, in Section 13.4). However, the method of subbagging wins the prize for the oddest sounding word. It is a combination of ‘subsample’

and ‘bagging,’ and it is the fairly obvious idea that you don’t need to produce samples that are the same size as the original data. If you make smaller datasets, then it makes sense to sample without replacement, but otherwise the implementation is only very slightly different from the bagging one, except that in NumPy you use `np.random.shuffle()` to produce the samples. It is common to use a dataset size that is half that of the original data, and the results of this can often be comparable to a full bagging simulation.

13.3 RANDOM FORESTS

If there is one method in machine learning that has grown in popularity over the last few years, then it is the idea of random forests. The concept has been around for longer than that, with several different people inventing variations, but the name that is most strongly attached to it is that of Breiman, who also described the CART algorithm that was discussed in Section 12.2, and also gave bagging its name.

The idea is largely that if one tree is good, then many trees (a forest) should be better, provided that there is enough variety between them. The most interesting thing about a random forest is the ways that it creates randomness from a standard dataset. The first of the methods that it uses is the one that we have just seen: bagging. If we wish to create a forest then we can make the trees different by training them on slightly different data, so we take bootstrap samples from the dataset for each tree. However, this isn’t enough randomness yet. The other obvious place where it is possible to add randomness is to limit the choices that the decision tree can make. At each node, a random subset of the features is given to the tree, and it can only pick from that subset rather than from the whole set.

As well as increasing the randomness in the training of each tree, it also speeds up the training, since there are fewer features to search over at each stage. Of course, it does introduce a new parameter (how many features to consider), but the random forest does not seem to be very sensitive to this parameter; in practice, a subset size that is the square root of the number of features seems to be common. The effect of these two forms of randomness is to reduce the variance without effecting the bias. Another benefit of this is that there is no need to prune the trees. There is another parameter that we don’t know how to choose yet, which is the number of trees to put into the forest. However, this is fairly easy to pick if we want optimal results: we can keep on building trees until the error stops decreasing.

Once the set of trees are trained, the output of the forest is the majority vote for classification, as with the other committee methods that we have seen, or the mean response for regression. And those are pretty much the main features needed for creating a random forest. The algorithm is given next before we see some results of using the random forest.

The Basic Random Forest Training Algorithm

- For each of N trees:
 - create a new bootstrap sample of the training set
 - use this bootstrap sample to train a decision tree
 - at each node of the decision tree, randomly select m features, and compute the information gain (or Gini impurity) only on that set of features, selecting the optimal one
 - repeat until the tree is complete
-

The implementation of this is very easy: we modify the decision to take an extra parameter, which is m , the number of features that should be used in the selection set at each stage. We will look at an example of using it shortly as a comparison to boosting.

Looking at the algorithm you might be able to see that it is a very unusual machine learning method because it is **embarrassingly parallel**: since the trees do not depend upon each other, you can both create and get decisions from different trees on different individual processors if you have them. This means that the random forest can run on as many processors as you have available with nearly linear speedup.

There is one more nice thing to mention about random forests, which is that with a little bit of programming effort they come with built-in test data: the bootstrap sample will miss out about 35% of the data on average, the so-called out-of-bootstrap examples. If we keep track of these datapoints then they can be used as novel samples for that particular tree, giving an estimated test error that we get without having to use any extra datapoints. This avoids the need for cross-validation.

As a brief example of using the random forest, we start by demonstrating that the random forest gets the correct results on the Party example that has been used in both this and the previous chapters, based on 10 trees, each trained on 7 samples, and with just two levels allowed in each tree:

RF prediction

```
['Party', 'Study', 'Party', 'Party', 'Pub', 'Party', 'Study', 'TV', 'Party', 'Study']
```

As a rather more involved example, the car evaluation dataset in the UCI Repository contains 1,728 examples aiming to classify whether or not a car is a good purchase based on six attributes. The following results compare a single decision tree, bagging, and a random forest with 50 trees, each based on 100 samples, and with a maximum depth of five for each tree. It can be seen that the random forest is the most accurate of the three methods.

Tree

```
Number correctly predicted 777.0
Number of testpoints 864
Percentage Accuracy 89.9305555556
```

```
Number of cars rated as good or very good 39.0
Number correctly identified as good or very good 18.0
Percentage Accuracy 46.1538461538
```

Bagger

```
Number correctly predicted 678.0
Number of testpoints 864
Percentage Accuracy 78.4722222222
```

```
Number of cars rated as good or very good 39.0
Number correctly identified as good or very good 0.0
Percentage Accuracy 0.0
```

```

Forest
Number correctly predicted 793.0
Number of testpoints 864
Percentage Accuracy 91.7824074074

Number of cars rated as good or very good 39.0
Number correctly identified as good or very good 20.0
Percentage Accuracy 51.28205128

```

13.3.1 Comparison with Boosting

There are some obvious similarities to boosting (Section 13.1), but it is the differences that are most telling. The most general thing is that boosting is exhaustive, in that it searches over the whole set of features at each stage, and each stage depends on the previous one. This means that boosting has to run sequentially, and the individual steps can be expensive to run. By way of contrast, the parallelism of the random forest and the fact that it only searches over a fairly small set of features at each stage speed the algorithm up a lot.

Since the algorithm only searches a small subset of the data at each stage, it cannot be expected to be as good as boosting for the same number of trees. However, since the trees are cheaper to train, we can make more of them in the same computational time, and often the results are amazingly good even on very large and complicated datasets.

In fact, the most amazing thing about random forests is that they seem to deal very well with really big datasets. It is fairly clear that they should do well computationally, since both the reduced number of features to search over and the ability to parallelise should help there. However, they seem to also produce good outputs based on surprisingly small parts of the problem space seen by each tree.

13.4 DIFFERENT WAYS TO COMBINE CLASSIFIERS

Bagging puts most of its effort into ensuring that the different classifiers see different data, since they see different samples of the data. This is different than boosting, where the data stays the same, but the importance of each datapoint changes for the different classifiers, since they each get different weights according to how well the previous classifiers have performed. Just as important for an ensemble method, though, is how it combines the outputs of the different classifiers. Both boosting and bagging take a vote from amongst the classifiers, although they do it in different ways: boosting takes a weighted vote, while bagging simply takes the majority vote. There are other alternatives to these methods, as well.

In fact, even majority voting is not necessarily simple. Some classification systems will only produce an output where all the classifiers agree, or more than half of them agree, whereas others simply take the most common output, which is what we usually mean by majority voting. The idea of not always producing an output is to ensure that the ensemble does not produce outputs that are contentious, because they are probably difficult datapoints. If the number of classifiers is odd and the classifiers are each independent of each other, then majority voting will return the correct label if more than half of the classifiers agree. Assuming that each individual classifier has a success rate of p , the probability of the ensemble getting the correct answer is a binomial distribution of the form:

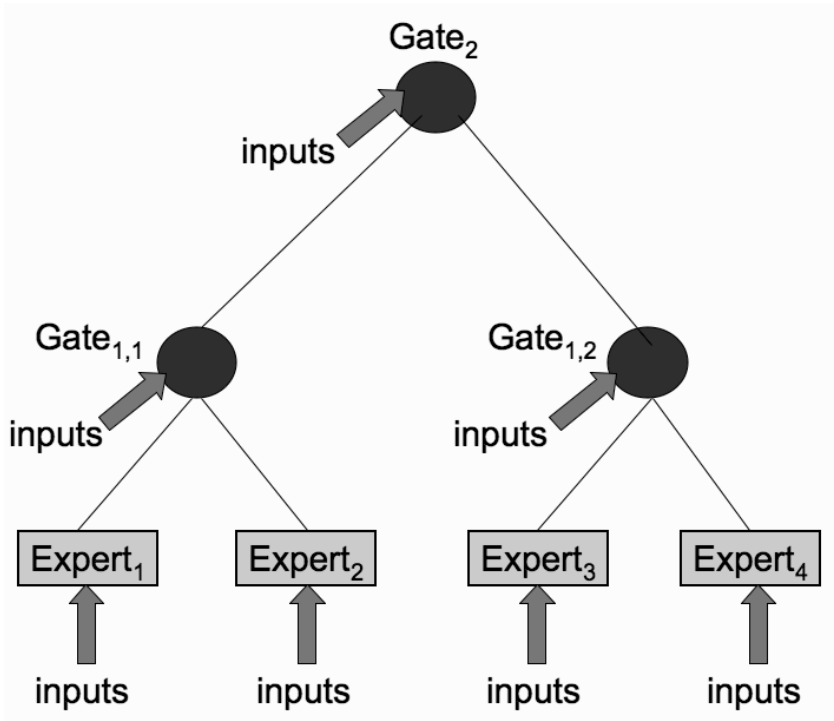


FIGURE 13.5 The Hierarchical Mixture of Networks network, consisting of a set of classifiers (experts) with gating systems that also use the inputs to decide which classifiers to trust.

$$\sum_{k=T/2+1}^T \binom{T}{k} p^k (1-p)^{T-k}, \quad (13.5)$$

where T is the number of classifiers. If $p > 0.5$, then this sum approaches 1 as $T \rightarrow \infty$. This is a lot of the power behind ensemble methods: even if each classifier only gets about half the answers right, if we use a decent number of classifiers (maybe 100), then the probability of the ensemble being correct gets close to 1. In fact, even with less than 50% chance of success for each individual classifier, the ensemble can often do very well indeed.

For regression problems, rather than taking the majority vote, it is common to take the mean of the outputs. However, the mean is heavily affected by outliers, with the result that the median is a more common average to use. It is the use of the median that produces the **bragging** algorithm, which is meant to imply ‘robust bagging’.

There is one more thing that can be done to combine classifiers, and that is to learn how to do it. There is an algorithm that does precisely this, known as the **mixture of experts**. Inputs are presented to the network, and each individual classifier makes an assessment. These outputs from the classifiers are then weighted by the relevant **gate**, which produces a weight w using the current inputs, and this is propagated further up the hierarchy. The most common version of the mixture of experts works as follows:

The Mixture of Experts Algorithm

- For each expert:
 - calculate the probability of the input belonging to each possible class by computing (where the $\mathbf{w}_i$ are the weights for that classifier):

$$o_i(\mathbf{x}, \mathbf{w}_i) = \frac{1}{1 + \exp(-\mathbf{w}_i \cdot \mathbf{x})}. \quad (13.6)$$

- For each gating network up the tree:
 - compute:

$$g_i(\mathbf{x}, \mathbf{v}_i) = \frac{\exp(\mathbf{v}_i \mathbf{x})}{\sum_l \exp(\mathbf{v}_l \mathbf{x})}. \quad (13.7)$$

- Pass as input to the next level gates (where the sum is over the relevant inputs to that gate):

$$\sum_k o_j g_j. \quad (13.8)$$

The most common way to train this network is using an **EM algorithm**. This is a general statistical approximation algorithm that is discussed in Section 7.1.1. It is also possible to use gradient descent on the parameters.

There are a couple of other ways to view these mixture of experts methods. One is to regard them as trees, except that the splits are not the **hard** splits that we performed in Chapter 12, but rather **soft**, because they are based on probability. The other is to compare them with radial basis function (RBF) networks (see Section 5.2). Each RBF gave a constant output within its receptive field. If, instead, each node were to give a linear approximation to the data, then the result would be the mixture of experts network.

FURTHER READING

Three papers that cover the three main ensemble methods described in this section are:

- R.E. Schapire. The boosting approach to machine learning: An overview. In D. D. Denison, M. H. Hansen, C. Holmes, B. Mallick, and B. Yu, editors, *Nonlinear Estimation and Classification*, Springer, Berlin, Germany, 2003.
- L. Breiman. Bagging predictors. *Machine Learning*, 26(2):123–140, 1996.
- M.I. Jordan and R.A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994.

An overview of the whole area is provided by:

- L. Kuncheva. *Combining Pattern Classifiers: Methods and Algorithms*. Wiley-Interscience, New York, USA, 2004.

For an alternative viewpoint, see:

- Sections 17.4 and 17.6–17.7 of E. Alpaydin. *Introduction to Machine Learning*, 2nd edition, MIT Press, Cambridge, MA, USA, 2009.

- Section 9.5 of R.O. Duda, P.E. Hart, and D.G. Stork. *Pattern Classification*, 2nd edition, Wiley-Interscience, New York, USA, 2001.

The original paper on Random Forests is still a very useful resource:
Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

PRACTICE QUESTIONS

Problem 13.1 Modify the decision tree implementation to use weights in the computation of the Gini impurity. This is not trivial, since you have to modify the total value of the Gini impurity, too. Once you have done it, use stump trees on the party data.

Problem 13.2 Implement the alternative form of boosting that uses the weights to sample the dataset. Does this make any difference to the outputs?

Problem 13.3 Stumping picks out the single most informative feature in the dataset and uses this. For a binary classification problem this will typically get at least half of the dataset correct. Why? How does this statement generalise to multiple classes?

Problem 13.4 Compare and contrast bagging and cross-validation.

Problem 13.5 The **Breastcancer** dataset in the UCI Machine Learning repository gives ten features and asks for a classification of breast tumours into benign and malignant. It is a difficult dataset, and provides a good comparison of the standard decision tree with boosted and bagged versions. Use all of the methods, using stumping and more advanced trees and see which work better.

Problem 13.6 The Mixture of Experts algorithm works with any kind of expert. Suppose that the experts were each MLPs. Implement this algorithm and see how well it does on the **Breastcancer** dataset above.

Problem 13.7 In Section 13.3 on the random forest, it was mentioned that there exists out-of-bootstrap data that can be used for validation and testing. Modify the code to keep track of this data.

Problem 13.8 Use the boosting code from Problem 13.2 above and compare it with the random forest on the cars dataset from the UCI Repository.